

SPA con React

1. Introducción a React



Taxonomía de las webs

Existen varios tipos de webs

Dependiendo de cómo usan las tecnologías
de **cliente** y **servidor**

Página web

Servidor estático

El servidor web sirve
contenido guardado en
el disco duro

Aplicación web

Servidor dinámico

El servidor web sirve
contenido generado
mediante código

Página web

Servidor estático

- Página web

Aplicación web

Servidor dinámico

- Aplicación web
- Aplicación web con AJAX
- Aplicación web SPA

Página web

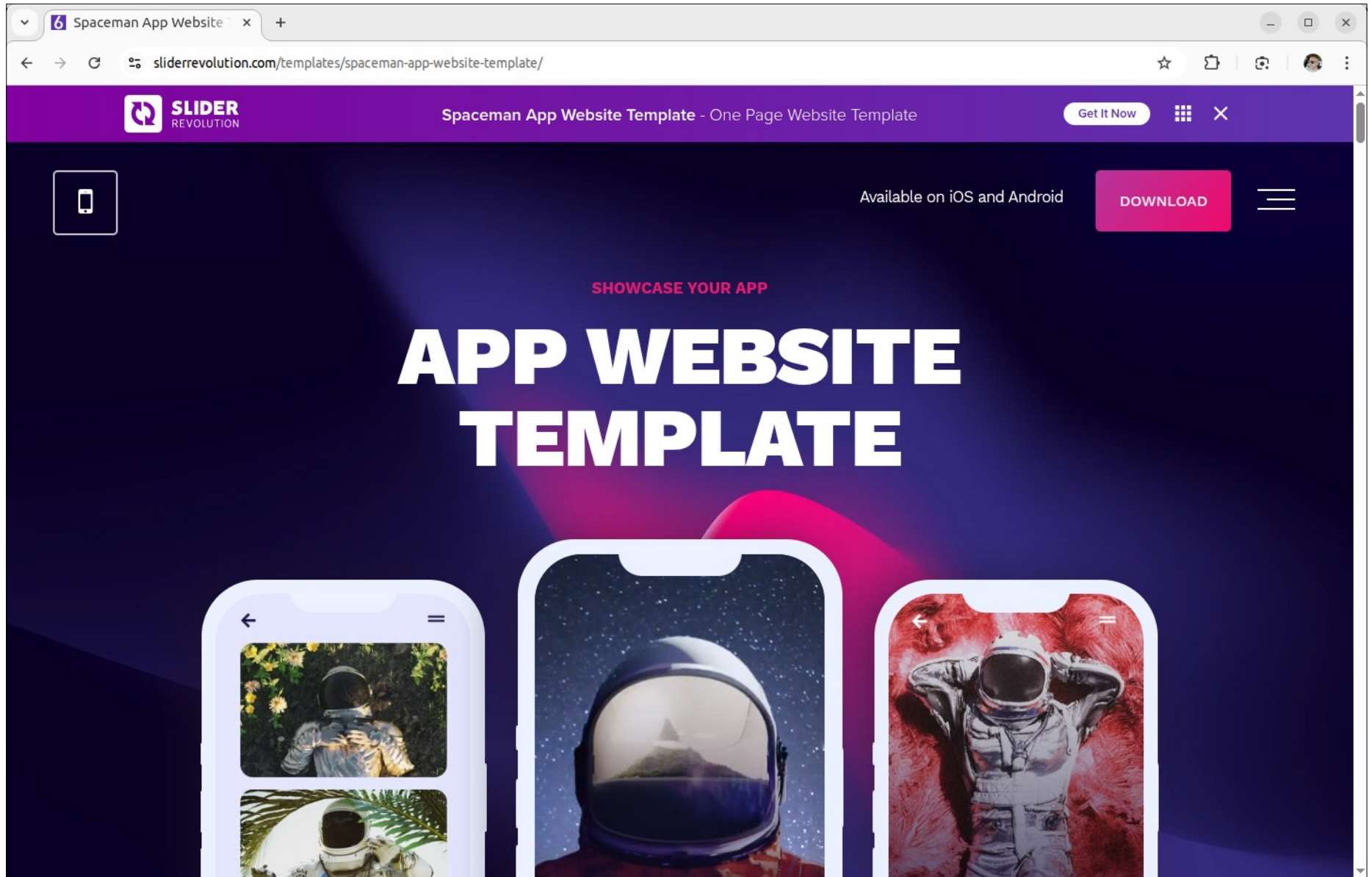
Servidor estático

- Página web

Aplicación web

Servidor dinámico

- Aplicación web
- Aplicación web con AJAX
- Aplicación web SPA



<https://www.sliderrevolution.com/templates/spaceman-app-website-template/>

Taxonomía de las webs

- **Página web**



- Se utilizan librerías de componentes CSS



tailwindcss



Bootstrap

Página web

Servidor estático

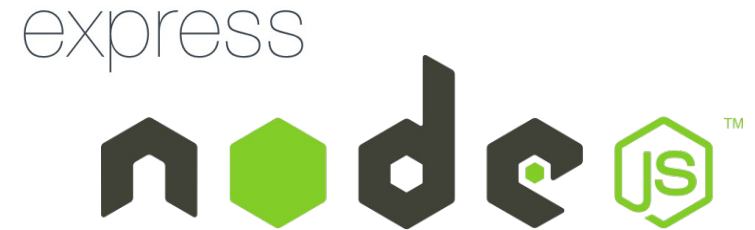
- Página web

Aplicación web

Servidor dinámico

- Aplicación web
- Aplicación web con AJAX
- Aplicación web SPA

Tecnologías de servidor



Inicio - Universidad Rey J x

urjc.es

Buscar...

INICIO EMPRESAS PRENSA SEDE ELECTRÓNICA CONTACTO AYUDA INTRANET TIENDA ONLINE

Universidad Rey Juan Carlos

UNIVERSIDAD ESTUDIOS ESTUDIAR EN LA URJC I+D+i INTERNACIONAL ACTUALIDAD

HERRAMIENTAS DE ACCESIBILIDAD

Feria Madrid es ciencia 2026 del 19 al 21 de marzo

El Museo Universidad de Navarra reconoce a Liuba González Cid

19-03-2026

Ángel Velázquez Iturbide, miembro del ACM Europe Awards Group

19-03-2026

Inaugurado el Bloomberg Lab en el campus de Madrid

18-03-2026

La URJC ha implantado oficialmente su

Lleno total en la jornada sobre empleo en salud

18-03-2026

La Universidad Rey Juan Carlos ha

create a new account

Your Email



Please use a valid email address.

Confirm Email

Country

United States



Zip Code



Please provide a valid Zip Code

Password



Your password must be between 6-16 characters long.

Confirm Password



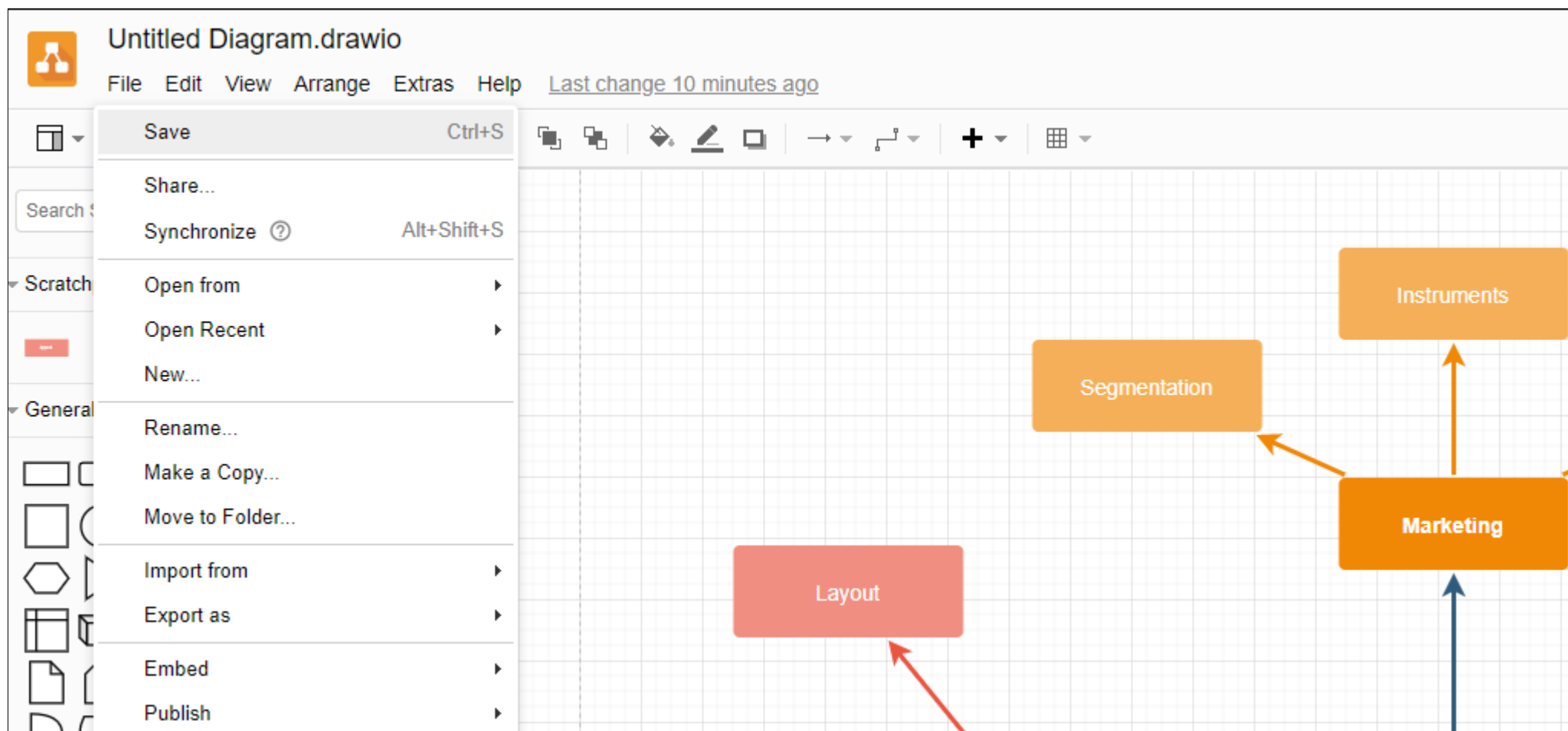
This does not match the password entered above.

☐ Yes, I agree to the [Terms of Use](#)

Sign Up

Why you'll love it

- See all your accounts in one place
- Set a budget and pay down your debt
- Find the best ways to grow your money
- Stay safe and secure



Página web

Servidor estático

- Página web

Aplicación web

Servidor dinámico

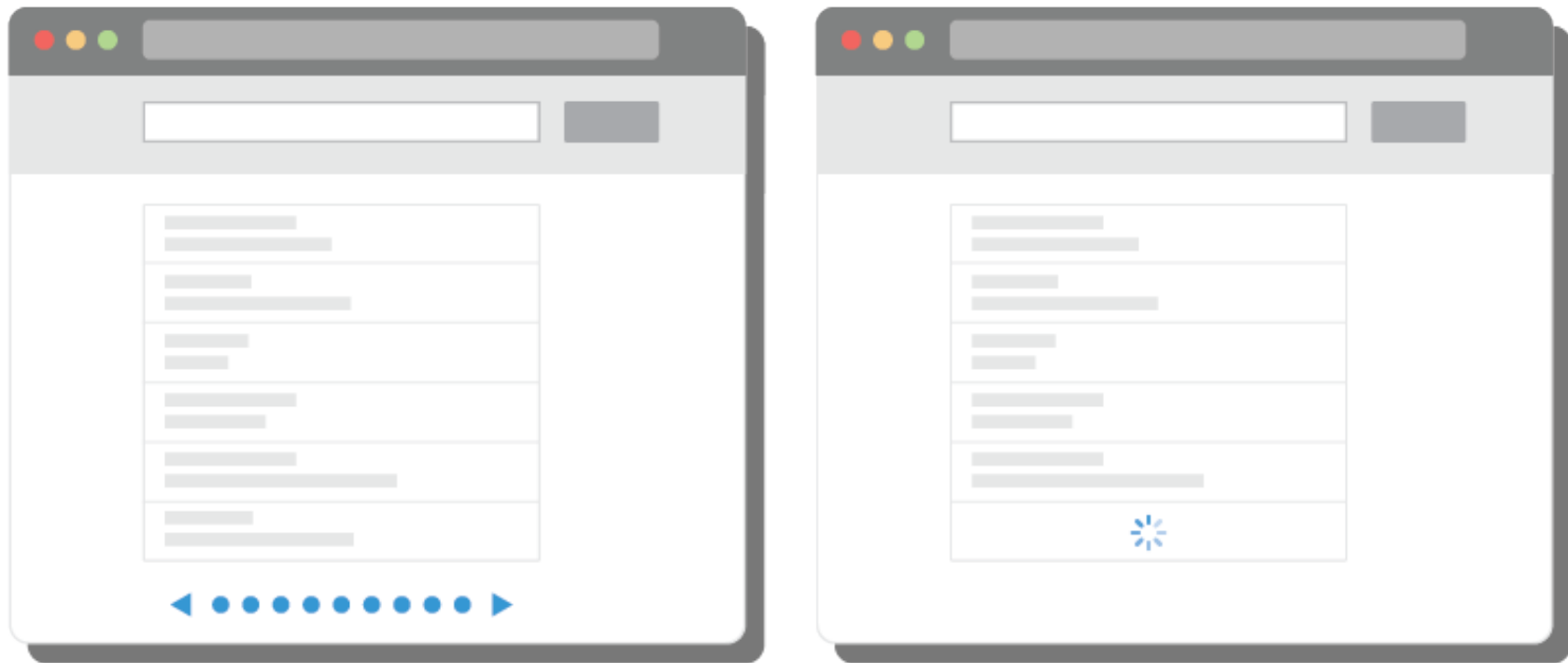
- Aplicación web
- Aplicación web con AJAX
- Aplicación web SPA

- **Aplicación web con AJAX**
 - Comunicación con el servidor en segundo plano (sin recarga completa de página)

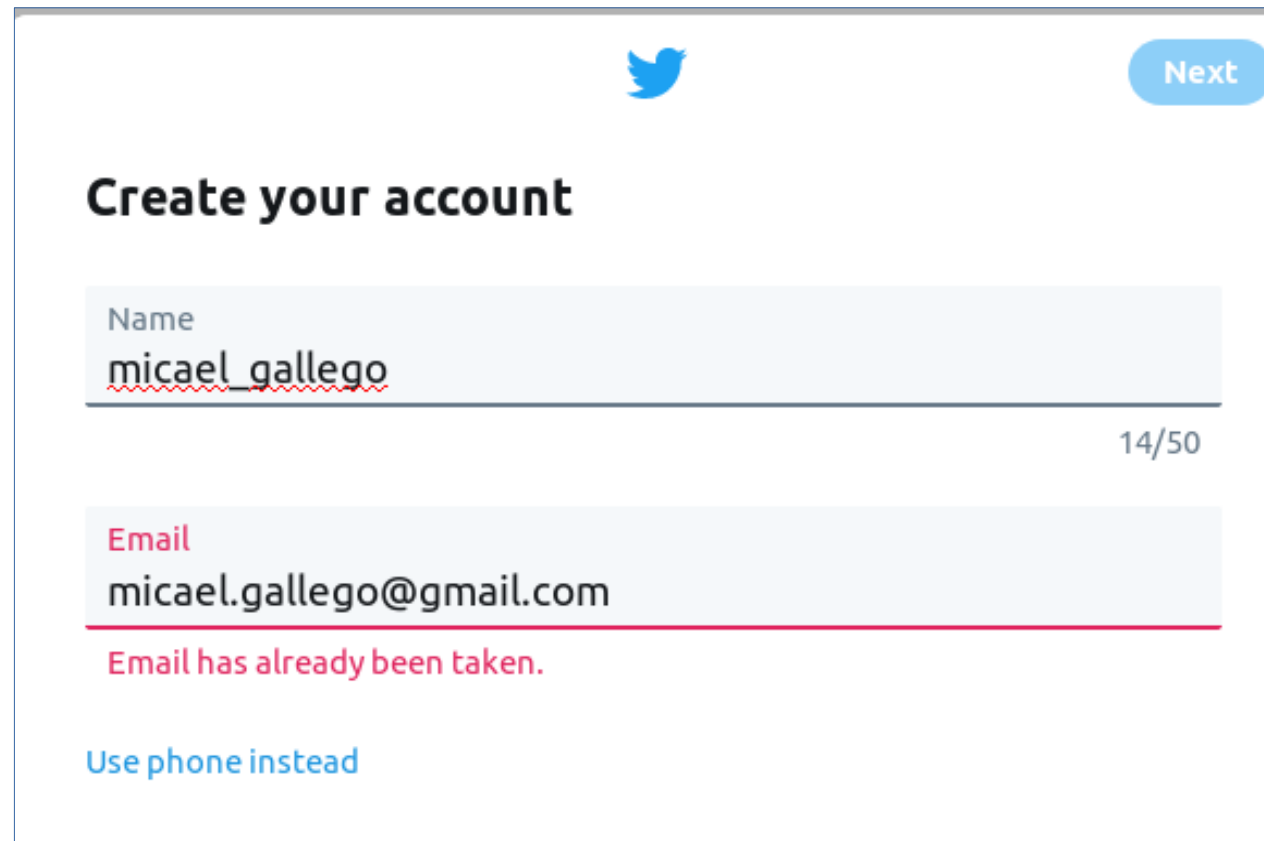


Taxonomía de las webs

- Aplicación web con AJAX



- Aplicación web con AJAX



A screenshot of the Twitter account creation form. At the top center is the Twitter bird logo. In the top right corner is a blue button labeled "Next". Below the logo is the heading "Create your account". There are two input fields. The first is labeled "Name" and contains the text "michael_gallego" with a red squiggly underline. To the right of this field is the character count "14/50". The second input field is labeled "Email" and contains the text "michael.gallego@gmail.com" with a red underline. Below the email field is a red error message: "Email has already been taken." At the bottom left of the form is a blue link that says "Use phone instead".

Página web

Servidor estático

- Página web

Aplicación web

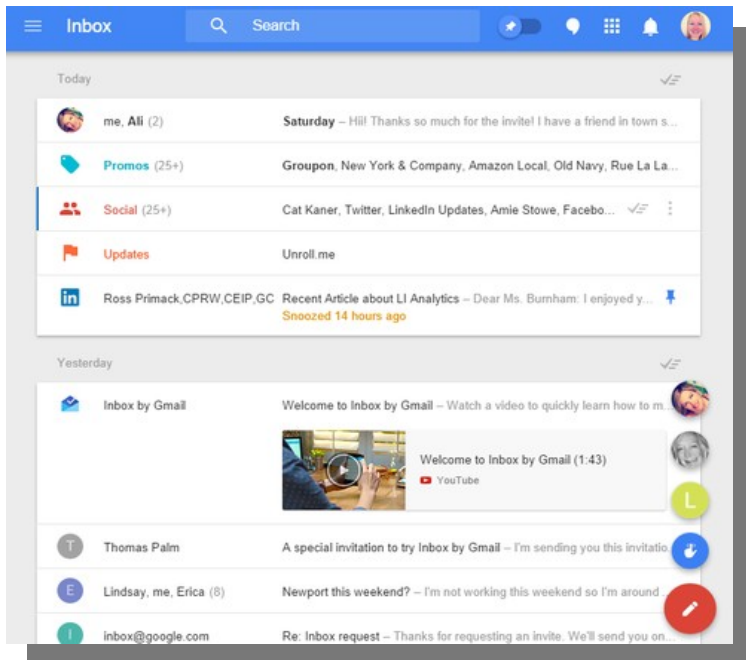
Servidor dinámico

- Aplicación web
- Aplicación web con AJAX
- Aplicación web SPA

- **Aplicación web SPA**
 - La página es un HTML casi vacío con JavaScript
 - El JavaScript construye la interfaz cargando datos del servidor (en segundo plano)
 - Mejor experiencia del usuario

Taxonomía de las webs

- Aplicación web SPA



1

`http://www.miweb.com/`



2

`http://api.miweb.com/users/34`

```
{  
  "name": "Pepe",  
  "surname": "López",  
  "age": 45,  
  "email": "pepe@miweb.com",  
  "friends": [ "María", "Juan" ]  
}
```



Google Maps

Universidad Rey Juan Carlos - Google Maps - Mozilla Firefox

Universidad Rey Juan Carlos x

https://www.google.es/maps/place/Universidad+Rey+Juan+Carlos/@40.3347481,-3.8755547, Buscar

an Carlos, Calle Tulipán, Móstoles

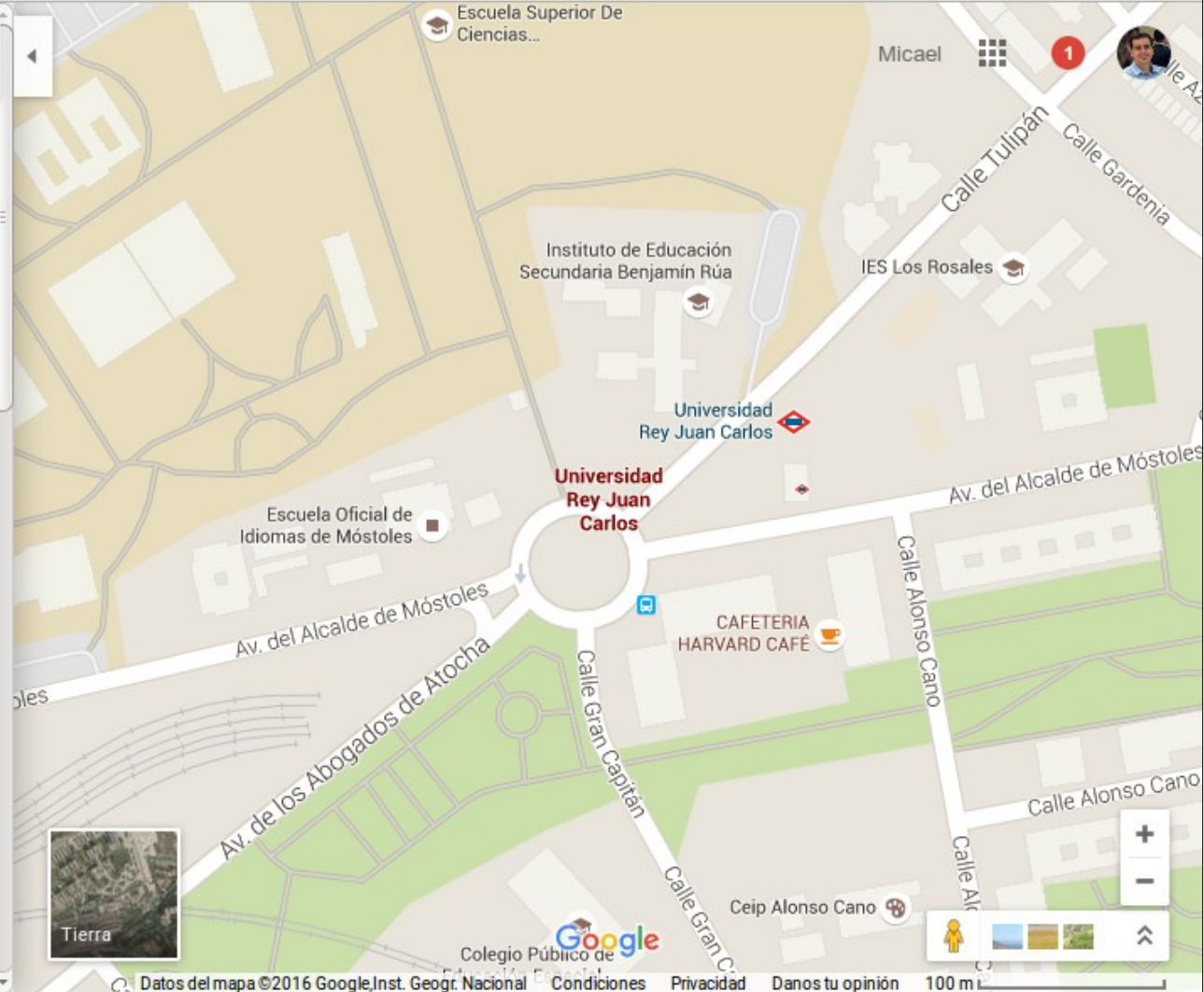


Universidad Rey Juan Carlos
3,8 ★★★★★ 66 reseñas
Universidad

Cómo llegar

GUARDAR LUGARES CERCANOS ENVIAR A TU TELÉFONO COMPARTIR

Calle Tulipán, s/n, 28933 Móstoles, Madrid
urjc.es
916 65 50 60
Cerrado hoy



Escuela Superior De Ciencias...

Micael

Calle Tulipán

Calle Gardenia

Instituto de Educación Secundaria Benjamín Rúa

IES Los Rosales

Universidad Rey Juan Carlos

Escuela Oficial de Idiomas de Móstoles

Av. del Alcalde de Móstoles

CAFETERIA HARVARD CAFÉ

Calle Alonso Cano

Av. de los Abogados de Atocha

Calle Gran Capitán

Ceip Alonso Cano

Colegio Público de

Tierra

Datos del mapa ©2016 Google, Inst. Geogr. Nacional Condiciones Privacidad Danos tu opinión 100 m

WhatsApp



projecto26/awesome-js

github.com/projecto26/awesome-jsgames

projecto26 / awesome-jsgames

Type / to search

+

🔍

🔗

📁

🔖

👤

<> Code

🔍 Issues 1

🔗 Pull requests 2

🔍 Actions

📁 Projects

🛡 Security

📊 Insights

 awesome-jsgames Public

🤍 Sponsor

👁 Watch 20

🔗 Fork 77

★ Star 837

main

1 Branch

0 Tags

Go to file

+

<> Code

 **jdnicollsc** Update README.md

35721e7 · 11 months ago

🕒 96 Commits

img	Add files via upload	5 years ago
CONTRIBUTING.md	Update CONTRIBUTING.md	3 years ago
LICENSE	Create LICENSE	5 years ago
README.md	Update README.md	11 months ago

📖 README

🔍 Contributing

📄 CC0-1.0 license

🔒 Security

Maintained? yes

website up

🔗 Follow @jdnicollsc

About

A curated list of awesome JavaScript Games 🎮

open-source

games

game-development

js13k

js13kgames

awesome-list

html5-game-development

javascript-game

js-games

javascript-games

javascript-game-engine

html5-games

games-list

awesome-lists

game-engines

html5-game-engine

js13kgames-starter

web-monetization

web-games

📖 Readme

📄 CC0-1.0 license

🔍 Contributing

Frameworks / librerías SPA



React

<https://react.dev/>

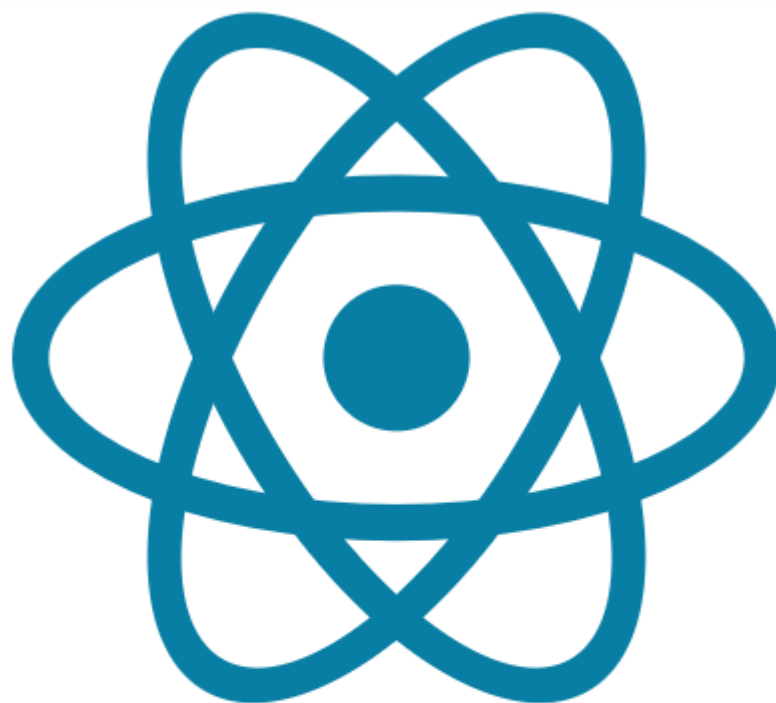


VUE.JS

<https://vuejs.org/>



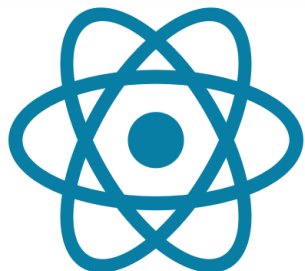
<https://angular.dev/>



React

<https://react.dev/>

- React es una librería para crear **webs dinámicas**
- Permite extender el **HTML con etiquetas propias**
 - Con contenido personalizado (HTML)
 - Con comportamiento personalizado (JavaScript)
- Interfaz basado en **componentes** (no en páginas)
- Se programa con **JavaScript** o **TypeScript** (una variante de JavaScript con tipado)



<https://react.dev/>



Características

- Generación de HTML partiendo de datos en variables JavaScript sin gestionar el DOM a bajo nivel
- Composición de componentes
- Compartir datos entre componentes
- Apps móviles (React Native)
- Es una librería muy ligera que se puede usar de muchas formas diferentes
- Hay un gran ecosistema alrededor

- React es una **librería** para añadir interactividad a una página
- Para crear aplicaciones SPA es necesario un **framework** para:
 - Gestión de páginas (rutas)
 - Gestiona la carga de datos de API REST
 - Generación del esqueleto del proyecto
 - Previsualización de cambios en el código al instante (*hot reloading*)

Frameworks React



NEXT.js



React Router

- **Framework** basado en React
- Simula páginas (rutas) en una aplicación SPA
- Gestiona la carga de datos de API REST
- Genera el esqueleto del proyecto
- Previsualiza los cambios en el código al instante (*hot reloading*) mediante Vite

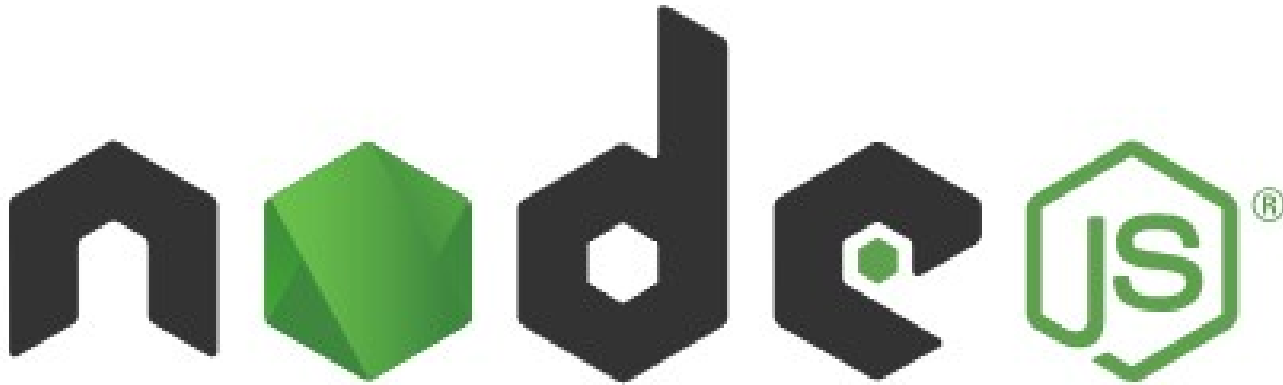


<https://reactrouter.com/>

Herramientas

Para desarrollar aplicaciones SPA

React Router



Download Node.js (LTS) 

<https://nodejs.org/>

React Router CLI

- Genera el **esqueleto** de una aplicación
- Permite añadir **componentes, servicios..**
- Permite **actualizar** código y previsualizar
- **Modo desarrollo:** Editar código y actualizar el navegador
- **Modo producción:** Optimiza el código para ejecución y rápida y poco tamaño (*build*)

<https://reactrouter.com/api/other-api/dev>

- Crear un proyecto

```
npx create-react-router@latest exam0_minimal --template remix-run/react-router-templates/minimal
```

```
• mica@mica-laptop:~/git/react$ npx create-react-router@latest ejem0 --template remix-run/react-router-templates/minimal

  create-react-router v7.13.0
  ■ Directory: Using ejem0 as project directory
  ■ Template: Using remix-run/react-router-templates/minimal...
  ✓ Template copied

  git Initialize a new git repository?
  Yes

  deps Install dependencies with npm?
  Yes

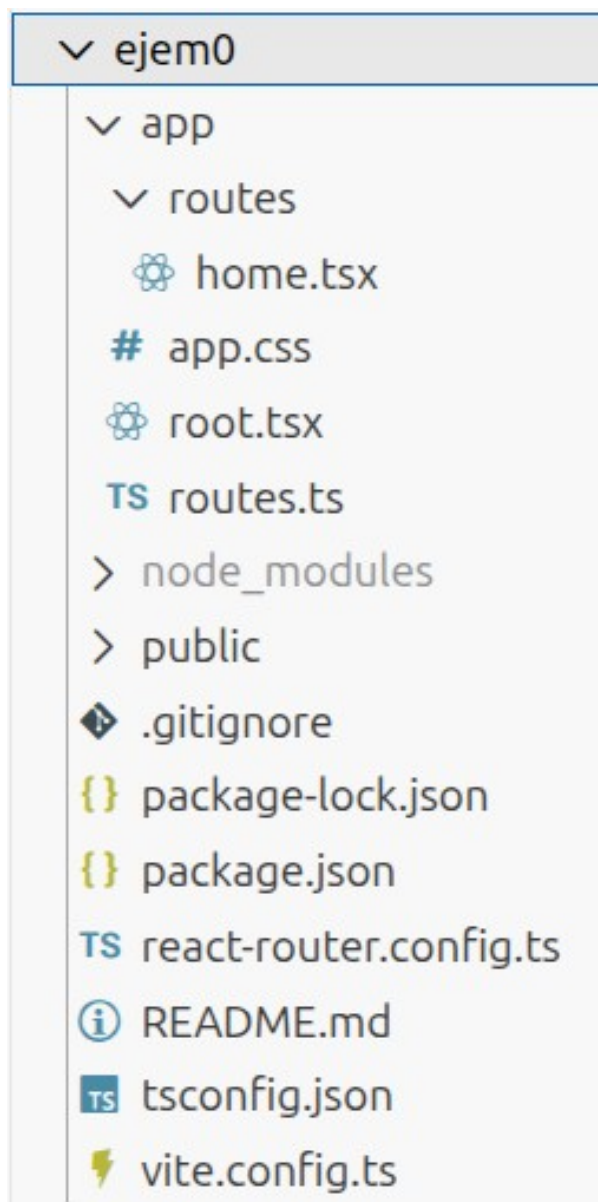
  ✓ Dependencies installed
  ✓ Git initialized

  done That's it!

  Enter your project directory using cd ./ejem0
  Check out README.md for development and deploy instructions.

  Join the community at https://rmx.as/discord
```

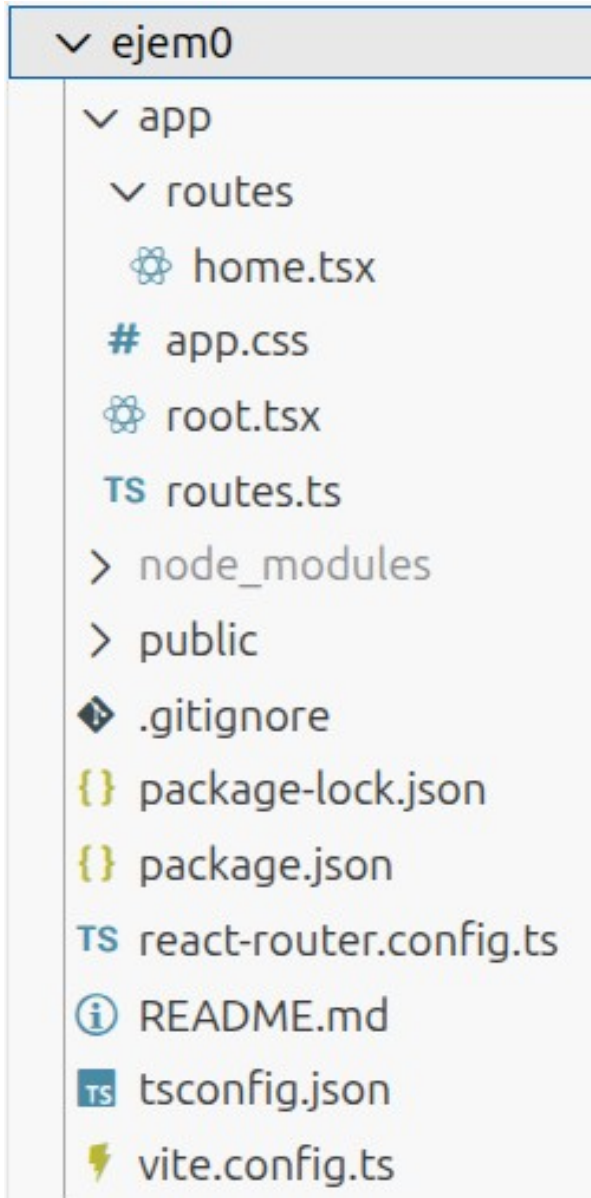
Un proyecto ocupa **108 MB** en disco



- **Ficheros/Carpetas generadas**

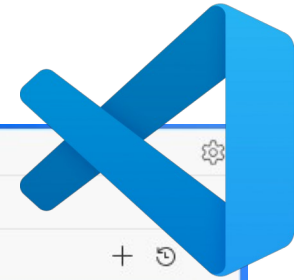
- app:** Código de la aplicación

- **public:** recursos estáticos web (imágenes...)
- **node_modules:** Librerías y herramientas descargadas
- **package.json:** Configuración de librerías Node y herramientas
- **react-router.config.json:** Configuración del framework
- **vite.config.ts:** Configuración de vite
- **tsconfig.json:** Configuración del compilador TypeScript



- **Ficheros/Carpetas generadas**
 - Ficheros que vamos a editar al principio
 - **home.tsx**: Código TypeScript app react
 - **app.css**: Estilos globales de la app
 - **public**: Recursos públicos (imágenes, etc.)
 - Ficheros que no vamos a editar (salvo casos puntuales)
 - **root.tsx**: Fichero principal de la aplicación
 - **react-router.config.ts**
 - **vite.config.ts**

Visual Studio Code

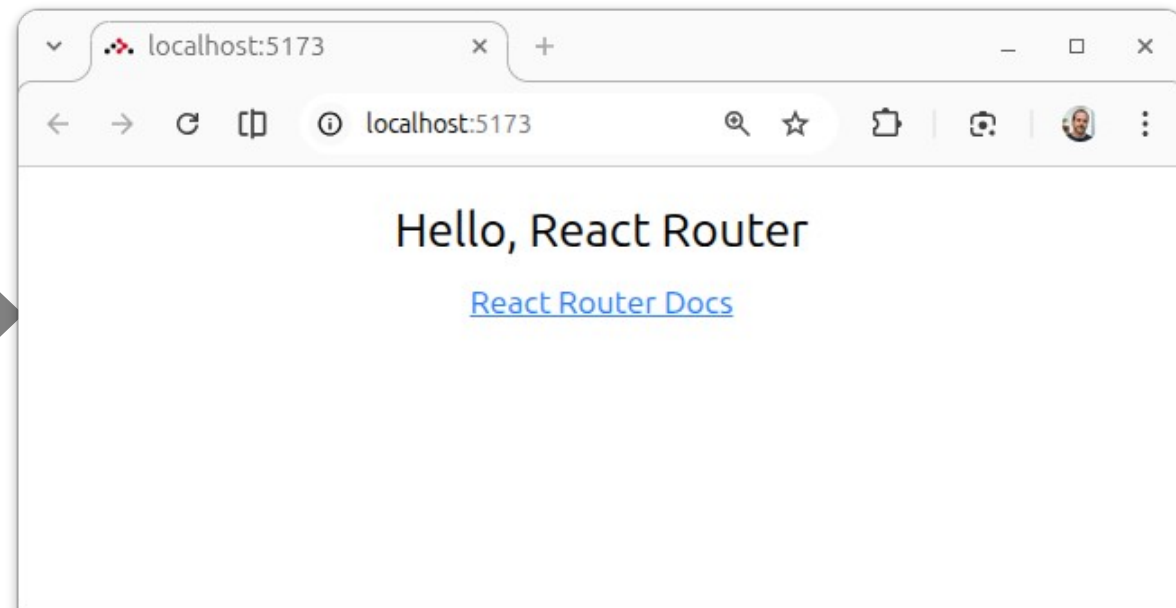


A screenshot of the Visual Studio Code interface. The left sidebar shows the 'AGENT SESSIONS' panel with sections for 'LOCAL CHAT AGENT', 'GITHUB COPILOT CLOUD AGENT', and 'GITHUB COPILOT CLI AGENT'. The 'GITHUB COPILOT CLOUD AGENT' section is active, showing a session titled 'Extract MailListItem into st...' with a duration of 10 minutes and a status of '#1 +42 -24'. Below this is a button labeled 'Start CLI Agent Session'. The main editor area displays two files: 'MailList.tsx' and 'MailListItem.tsx'. The 'MailList.tsx' file is open, showing a TypeScript function 'MailList()' that uses 'solid-js' and 'solid-router' hooks. The function has parameters 'mailbox' and 'id', and it returns a list of email items. The 'MailListItem.tsx' file is also open, showing a component that renders a list item. The right sidebar shows the 'CHAT' panel with a message from the 'GitHub Copilot Cloud Agent' asking to 'Please update the MailList component to abstract mail list items into their own standalone component'. Below this is a section for 'kenzi.lawson' who selected 'Delegate'. Another message from the 'GitHub Copilot Cloud Agent' is shown, stating '[WIP] Refactor MailList to abstract mail list items - Copilot'. At the bottom of the chat panel, there is a section for 'Describe what to build next' with a dropdown menu for 'Agent' and 'Claude Opus 4.6'.

<https://code.visualstudio.com>

- Empezar a desarrollar

```
$ cd exam0_minimal  
$ npm run dev
```



<http://localhost:5173/>

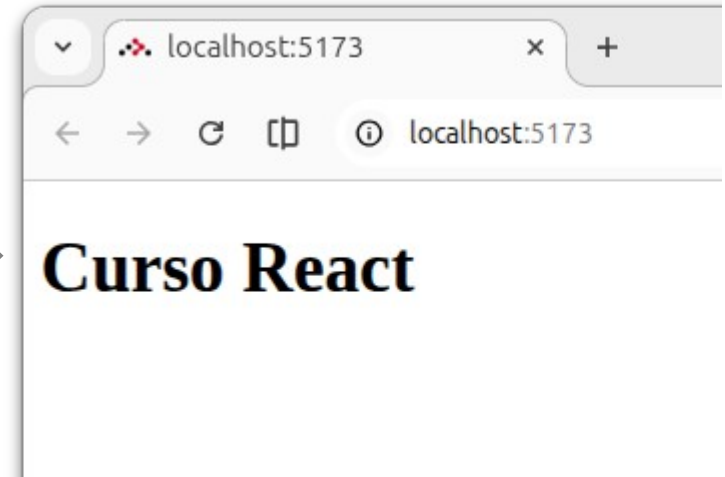
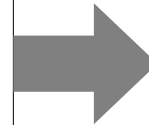
- Ejecutar la app en modo desarrollo

```
npm run dev
```

Al cambiar un fichero fuente y guardar, la aplicación se **actualiza automáticamente**

exam0_minimal/app/routes/home.tsx

```
export default function Home() {  
  return (  
    <h1>Curso React</h1>  
  );  
}
```



- Ejecutar el ejemplos y ejercicios
 - Descargar el fichero .zip
 - Descomprimir en una carpeta y abrir en VSCode
 - Por cada proyecto
 - Descarga las librerías del proyecto

```
$ cd exam1_render_var  
$ npm install
```

- Ejecutar la aplicación en modo desarrollo

```
$ npm run dev
```

TypeScript

- React se puede programar con **JavaScript** o **TypeScript**
- En los proyectos **React Router**, por defecto se usa TypeScript
- Es una variante de JavaScript cada vez más **popular**



Características

- Variante de JavaScript con **validación de tipos**
- Se usa cada vez más por la ayuda del IDE (**autocompletar y avisa de errores**)
- Integrado en todos los **editores / IDEs**

<http://www.typescriptlang.org/>



```
export class Empleado {                                     TypeScript

    private nombre:string;
    private salario:number;

    constructor(nombre:string, salario:number){
        this.nombre = nombre;
        this.salario = salario;
    }

    getNombre(){
        return this.nombre;
    }

    toString(){
        return "Nombre:"+this.nombre+
            ", Salario:"+this.salario;
    }
}
```

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:"+nombre+  
            ", Salario:"+salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:"+this.nombre+  
            ", Salario:"+this.salario;  
    }  
}
```

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:"+nombre+  
            ", Salario:"+salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:"+this.nombre+  
            ", Salario:"+this.salario;  
    }  
}
```

En Java las clases son públicas. En TypeScript las clases se exportan

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:"+nombre+  
            ", Salario:"+salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:"+this.nombre+  
            ", Salario:"+this.salario;  
    }  
}
```

En TS los tipos se ponen tras el nombre con : (dos puntos)

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:"+nombre+  
            ", Salario:"+salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:"+this.nombre+  
            ", Salario:"+this.salario;  
    }  
}
```

En TS el constructor es "constructor"

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:"+nombre+  
            ", Salario:"+salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:"+this.nombre+  
            ", Salario:"+this.salario;  
    }  
}
```

En TS los tipos se infieren (si quieres), incluso en el tipo que devuelven los métodos

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:" + nombre +  
            ", Salario:" + salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:" + this.nombre +  
            ", Salario:" + this.salario;  
    }  
}
```

En TS siempre que quieras acceder a un elemento de la clase tendrás que usar this.

TypeScript

```
import { Empleado } from "../Empleado";  
  
let emps = new Array<Empleado>();  
  
emps.push(new Empleado('Pepe', 500));  
emps.push(new Empleado('Juan', 200));  
  
for(let emp of emps){  
    console.log(emp.getNombre());  
}  
  
empleados.forEach(emp => {  
    console.log(emp);  
});
```

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();

emps.add(new Empleado('Pepe', 500));
emps.add(new Empleado('Juan', 200));

for(Empleado emp : emps){
    System.out.println(emp.getNombre());
}

empleados.forEach(emp -> {
    System.out.println(emp);
});
```

TypeScript

```
import { Empleado } from "../Empleado";

let emps = new Array<Empleado>();

emps.push(new Empleado('Pepe', 500));
emps.push(new Empleado('Juan', 200));

for(let emp of emps){
    console.log(emp.getNombre());
}

empleados.forEach(emp => {
    console.log(emp);
});
```

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();

emps.add(new Empleado('Pepe', 500));
emps.add(new Empleado('Juan', 200));

for(Empleado emp : emps){
    System.out.println(emp.getNombre());
}

empleados.forEach(emp -> {
    System.out.println(emp);
});
```

TypeScript

```
import { Empleado } from "./Empleado";

let emps = new Array<Empleado>();

emps.push(new Empleado('Pepe', 500));
emps.push(new Empleado('Juan', 200));

for(let emp of emps){
    console.log(emp.getNombre());
}

empleados.forEach(emp => {
    console.log(emp);
});
```

En Java las clases de la misma carpeta son del mismo paquete.
En TS cada fichero es un módulo, por eso hay que importar desde otros ficheros

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();  
  
emps.add(new Empleado('Pepe', 500));  
emps.add(new Empleado('Juan', 200));  
  
for(Empleado emp : emps){  
    System.out.println(emp.getNombre());  
}  
  
empleados.forEach(emp -> {  
    System.out.println(emp);  
});
```

TypeScript

```
import { Empleado } from "../Empleado";  
  
let emps = new Array<Empleado>();  
  
emps.push(new Empleado('Pepe', 500));  
emps.push(new Empleado('Juan', 200));  
  
for(let emp of emps){  
    console.log(emp.getNombre());  
}  
  
empleados.forEach(emp => {  
    console.log(emp);  
});
```

En TS las variables se declaran con let y tienen ámbito de bloque y no se pueden declarar dos veces. Se podría usar var (como JS), pero el ámbito sería la función y se podrían redeclarar

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();  
  
emps.add(new Empleado('Pepe', 500));  
emps.add(new Empleado('Juan', 200));  
  
for(Empleado emp : emps){  
    System.out.println(emp.getNombre());  
}  
  
empleados.forEach(emp -> {  
    System.out.println(emp);  
});
```

TypeScript

```
import { Empleado } from "../Empleado";  
  
let emps = new Array<Empleado>();  
  
emps.push(new Empleado('Pepe', 500));  
emps.push(new Empleado('Juan', 200));  
  
for(let emp of emps){  
    console.log(emp.getNombre());  
}  
  
empleados.forEach(emp => {  
    console.log(emp);  
});
```

En TS las variables se pueden declarar con tipo (después de :), pero es opcional porque el tipo se infiere del valor con el que se inicializan

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();
```

```
emps.add(new Empleado('Pepe', 500));  
emps.add(new Empleado('Juan', 200));
```

```
for(Empleado emp : emps){  
    System.out.println(emp.getNombre());  
}
```

```
empleados.forEach(emp -> {  
    System.out.println(emp);  
});
```

TypeScript

```
import { Empleado } from "../Empleado";
```

```
let emps = new Array<Empleado>();
```

```
emps.push(new Empleado('Pepe', 500));  
emps.push(new Empleado('Juan', 200));
```

```
for(let emp of emps){  
    console.log(emp.getNombre());  
}
```

```
empleados.forEach(emp => {  
    console.log(emp);  
});
```

En Java usamos List y ArrayList generificados del API.
En TS usamos el Array nativo de JavaScript generificado por TS.

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();  
emps.add(new Empleado('Pepe', 500));  
emps.add(new Empleado('Juan', 200));  
  
for(Empleado emp : emps){  
    System.out.println(emp.getNombre());  
}  
  
empleados.forEach(emp -> {  
    System.out.println(emp);  
});
```

TypeScript

```
import { Empleado } from "../Empleado";  
  
let emps = new Array<Empleado>();  
emps.push(new Empleado('Pepe', 500));  
emps.push(new Empleado('Juan', 200));  
  
for(let emp of emps){  
    console.log(emp.getNombre());  
}  
  
empleados.forEach(emp => {  
    console.log(emp);  
});
```

En Java List el método es "add"
En el array de JavaScript el método es "push"

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();

emps.add(new Empleado('Pepe', 500));
emps.add(new Empleado('Juan', 200));

for(Empleado emp : emps){
    System.out.println(emp.getNombre());
}

empleados.forEach(emp -> {
    System.out.println(emp);
});
```

TypeScript

```
import { Empleado } from "../Empleado";

let emps = new Array<Empleado>();

emps.push(new Empleado('Pepe', 500));
emps.push(new Empleado('Juan', 200));

for(let emp of emps){
    console.log(emp.getNombre());
}

empleados.forEach(emp => {
    console.log(emp);
});
```

La sintaxis del foreach es muy parecida en Java y TS.
En TS está basado en iteradores (como en Java)

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();

emps.add(new Empleado('Pepe', 500));
emps.add(new Empleado('Juan', 200));

for(Empleado emp : emps){
    System.out.println(emp.getNombre());
}

empleados.forEach(emp -> {
    System.out.println(emp);
});
```

TypeScript

```
import { Empleado } from "../Empleado";

let emps = new Array<Empleado>();

emps.push(new Empleado('Pepe', 500));
emps.push(new Empleado('Juan', 200));

for(let emp of emps){
    console.log(emp.getNombre());
}

empleados.forEach(emp => {
    console.log(emp);
});
```

Las expresiones lambda de Java se llaman *arrow functions* en TS.
Se diferencian en la "flecha" con – o con =

Objetos literales “tipados”

```
interface SquareConfig {  
  color: string;  
  width?: number;  
}
```

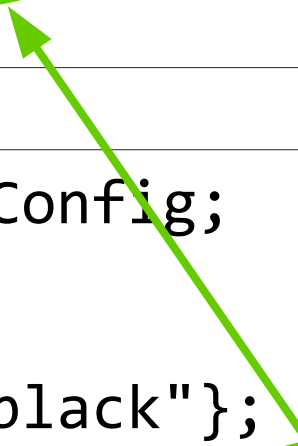
```
let config: SquareConfig;  
  
config = {color: "black"};  
config = {color: "black", width: 20};
```

Objetos literales “tipados”

```
interface SquareConfig {  
  color: string;  
  width?: number;  
}
```

```
let config: SquareConfig;
```

```
config = {color: "black"};  
config = {color: "black", width: 20};
```



Tipos unión

```
let id: string | number;  
  
id = 3;  
...  
Id = "3";  
  
if(typeof id === "string"){  
    ...  
}
```

TypeScript permite que una variable tenga valores de diferentes tipos